

SDE-1 Weekend Assignment – Build a Low-Code API Orchestration Platform

Objective

Modern enterprises integrate with multiple third-party APIs (banks, government systems, payment gateways, KYC providers, etc.). Developing and maintaining custom integration code for every provider is expensive and difficult to scale.

Your task is to build a **configuration-driven API orchestration platform** that allows users to expose their own REST APIs without writing business logic for each integration.

The platform should allow a user to define an API using configuration (JSON/YAML/Database/UI), map request and response fields, invoke one or more downstream APIs, transform data, and return the final response.

The goal is to demonstrate how software can be made configurable, reusable, and extensible rather than hardcoding every integration.

Problem Statement

Build a platform that enables users to:

1. Define an API endpoint using configuration.
2. Accept incoming REST requests.
3. Validate incoming payloads.
4. Map request fields to downstream vendor APIs.
5. Invoke one or more external/mock APIs.
6. Transform or combine responses.
7. Return a standardized response.
8. Configure the workflow without changing application code.

For example,

Client calls

POST /verify-pan

Your platform should determine (from configuration):

- ☐ Which vendor API(s) to invoke
- ☐ How to map incoming fields
- ☐ How to authenticate
- ☐ Which headers to send
- ☐ How to transform the vendor response
- ☐ What final response should be returned

without requiring code changes.

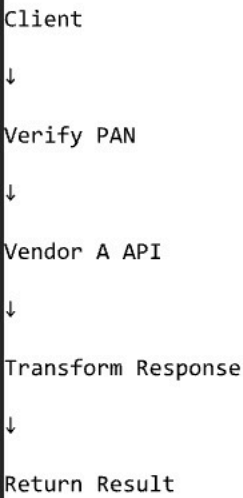
Functional Requirements

Your solution should support:

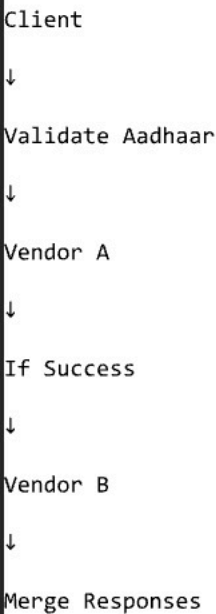
- ☐ Dynamic API creation through configuration
- ☐ Request validation
- ☐ Request/response field mapping
- ☐ HTTP API invocation
- ☐ Multiple API orchestration
- ☐ Conditional execution
- ☐ Error handling
- ☐ Retry mechanism
- ☐ Standardized response format
- ☐ Execution logging

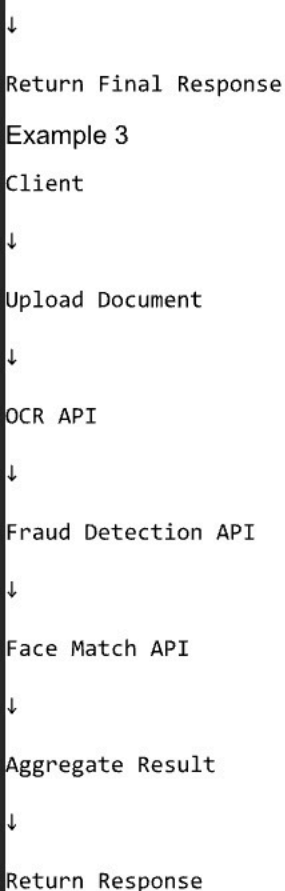
Example Use Cases

Example 1



Example 2





Technical Expectations

You are free to choose your technology stack.

Your solution should demonstrate:

- ☐ Clean architecture
- ☐ Extensible design
- ☐ Good coding practices
- ☐ Proper API design
- ☐ Logging
- ☐ Error handling
- ☐ Documentation

Deliverables

- ☐ Source code
- ☐ README
- ☐ Sample configurations
- ☐ API documentation

- ☐ Architecture diagram
- ☐ Sample requests and responses

Bonus Features (Optional)

Additional points will be awarded for implementing any of the following:

- ☐ Visual workflow editor
- ☐ Authentication (JWT/API Key)
- ☐ Rate limiting
- ☐ Versioned APIs
- ☐ Metrics endpoint
- ☐ Docker support
- ☐ Swagger/OpenAPI
- ☐ Workflow versioning
- ☐ Parallel execution
- ☐ Webhook support
- ☐ Scheduled execution
- ☐ Caching
- ☐ Plugin architecture
- ☐ Kubernetes deployment
- ☐ CI/CD pipeline

Agentic AI Bonus ☐

Additional bonus points will be awarded if your solution includes an AI Agent.

Example capabilities include:

- ☐ Convert natural language into API workflow configuration.

Example:

Input:

Create an API that validates a PAN using Vendor A and, if successful, fetches GST details from Vendor B.

The agent generates the required orchestration configuration automatically.

OR

- ☐ Generate request/response field mappings automatically.

OR

- ☐ Recommend API orchestration improvements.

OR

- ☐ Detect invalid workflow configurations.

OR

- ☐ Generate test cases for the configured APIs.

Creativity is encouraged. Surprise us with what your agent can do.

Evaluation Criteria

We are primarily evaluating:

- ☐ Engineering fundamentals
- ☐ Problem solving
- ☐ Software design
- ☐ Extensibility
- ☐ Code quality
- ☐ API design
- ☐ Documentation
- ☐ Creativity
- ☐ AI-assisted engineering (Bonus)

We value thoughtful engineering decisions over the number of features implemented.